Desenvolvimento de Aplicações Interativas com React [26E4_2]

RELATÓRIO TÉCNICO DE PROJETO

Sistema Gerenciador de Estoque com React e Spring Boot

Identificação do Autor e Disciplina	
Autor / Discente:	Francisco Tarcizo Façanha Neto
Disciplina:	Desenvolvimento de Aplicações Interativas com React [26E4_2]
Finalidade:	Avaliação e Conclusão de Módulo de Pós-Graduação
Repositório	https://github.com/tarcizo12/Desenvolvimento-de-aplicacoes-Interativas-com-React-26E4_2-.git

1. Título do Projeto

Gerenciador de Estoque — Aplicação web full-stack desenvolvida para a organização, catalogação e controle operacional de produtos, categorias e fornecedores.

2. Objetivo do Projeto

O projeto oferece uma interface corporativa e reativa para manter um catálogo de produtos e suas informações de estoque, relacionando cada produto a uma categoria temática e a um fornecedor homologado. O sistema resolve o problema de fragmentação e controle manual de inventário ao centralizar a gestão de itens perecíveis e não perecíveis.

Adicionalmente, a aplicação permite cadastrar fornecedores individualmente ou importar seus dados de forma ágil e síncrona a partir de um arquivo CSV. A API REST valida rigorosamente as regras de domínio e persiste as entidades em banco de dados relacional MySQL.

3. Features Implementadas

Gestão de Produtos (Catálogo e Estoque):
Operações completas de CRUD (listar, cadastrar, editar e excluir com modal de confirmação). O modelo contempla diferenciação estrutural e polimórfica entre produtos perecíveis (controle de data de validade e número de lote) e produtos não perecíveis (parametrização de meses de garantia comercial).

Categorias de Produtos:
Listagem tabular e cadastro de categorias temáticas para classificar, organizar e filtrar os itens do catálogo de estoque.

Gestão de Fornecedores e Importação em Lote:
Listagem e cadastro individual de fornecedores com validação de campos corporativos (CNPJ, telefone, e-mail, endereço). Importação síncrona via upload de arquivo CSV, informando no retorno a quantidade de registros inseridos e duplicados desconsiderados.

Consulta Externa e Preenchimento Automático de CEP:
O formulário de cadastro de fornecedor integra com o Web Service ViaCEP via Fetch API para localização e autopreenchimento de endereço, tratando estados de consulta, CEP inexistente e possíveis falhas de conexão.

Interface Reativa e Experiência do Usuário (UX):
Formulários controlados, abas de navegação intuitivas, confirmação de exclusão e tratamento visual explícito para estados de carregamento (spinners), sucesso e erro.

Gerenciamento de Estado Global e Cache:
Redux Toolkit armazena a sessão demonstrativa do operador; TanStack Query (React Query) mantém cache de requisições e sincroniza dados em segundo plano com a API REST.

Navegação Declarativa e Sessão Demonstrativa:
Páginas organizadas com React Router DOM. O acesso inicial por nome de operador configura uma proteção demonstrativa do frontend (apresentando o nome no cabeçalho), não constituindo autenticação real de segurança.

Persistência Relacional e Validações de Domínio:
API REST em Spring Boot com validações de payload (Bean Validation), regras de domínio com herança de entidades e persistência em MySQL 8.4.

4. Tecnologias Utilizadas

A arquitetura da solução está dividida em camadas bem definidas e modernas:

Camada	Tecnologias / Bibliotecas	Finalidade no Projeto
Frontend	React 18, TypeScript, Create React App, React Router DOM, Redux Toolkit, TanStack Query, Material UI, Fetch API, CSS	Construção da interface de usuário reativa, tipagem estática, rotas, cache de requisições e gerenciamento de estado.
Backend	Java 21, Spring Boot, Spring Web, Spring Validation, Spring Data JPA, OpenFeign, Springdoc OpenAPI (Swagger)	Serviços RESTful, regras de negócio, persistência relacional, validação de dados e documentação interativa da API.
Banco de Dados	MySQL 8.4	Armazenamento transacional dos dados com integridade referencial.
Conteinerização	Docker, Docker Compose, Nginx, Node.js 22	Orquestração multi-container para execução isolada e servidor estático Nginx para o build do frontend.
Testes	Jest, JUnit 5, Mockito, Spring Boot Test	Testes unitários e de integração no frontend e no backend.

5. Instruções de Execução

5.1. Preparação e Requisitos

O frontend foi construído utilizando Create React App com testes automatizados em Jest. Para a execução completa via Docker Compose, são necessários apenas:

• Docker e Docker Compose instalados e em execução (Docker Desktop no Windows/macOS; Docker Engine + Compose no Linux).
• Git para clonar o repositório.
• Não é obrigatório instalar Node.js ou Java na máquina para a execução conteinerizada via scripts.

5.2. Procedimento de Inicialização

Passo 1 — Clonar o Repositório:

```
git clone
https://github.com/tarcizo12/Desenvolvimento-de-aplicacoes-Interativas-com-React-26E4_2-
.git
cd Desenvolvimento-de-aplicacoes-Interativas-com-React-26E4_2-
```

Passo 2 — Inicialização no Windows:

Com o Docker Desktop aberto, execute o script que sobe o backend, o banco e o frontend em sequência:

```
scripts\iniciar-tudo.bat
```

(Se preferir iniciar separadamente: 'scripts\windows\iniciar-backend.bat' e 'scripts\windows\iniciar-frontend.bat').

Passo 3 — Inicialização no Linux / macOS:

Conceda permissão aos scripts e execute:

```
chmod +x scripts/*.sh scripts/linux/*.sh
./scripts/iniciar-tudo.sh
```

5.3. Endereços e Portas de Acesso

Serviço	Endereço Local	Observações
Interface Web (React)	http://localhost:8082	Servida em container Nginx (proxy API)
API REST (Spring Boot)	http://localhost:8080	Endpoints da aplicação backend
Documentação Swagger	http://localhost:8080/swagger-ui.html	Interface interativa para teste da API
Banco MySQL	localhost:3306	Base inicializada com init-dev.sql

5.4. Procedimento para Encerrar

• Windows: 'scripts\windows\parar-aplicacao.bat'
• Linux/macOS: './scripts/linux/parar-aplicacao.sh'

Por padrão, os scripts preservam os volumes do banco de dados MySQL.

5.5. Principais Endpoints da API

• Produtos: '/com/api/produtos' (CRUD completo).
• Categorias: '/com/api/categorias' (listagem e cadastro).
• Fornecedores: '/com/api/fornecedores' (listagem e cadastro).
• Importação CSV: 'PUT /com/api/fornecedores/importacao' (multipart field 'arquivo').

6. Link do Repositório GitHub

O código-fonte completo da aplicação, histórico de desenvolvimento e configurações Docker estão disponíveis em:
https://github.com/tarcizo12/Desenvolvimento-de-aplicacoes-Interativas-com-React-26E4_2-

7. Screenshots e Demonstração

As imagens a seguir ilustram as principais telas da aplicação e o modelo de domínio implementado:

7.1. Acesso Demonstrativo

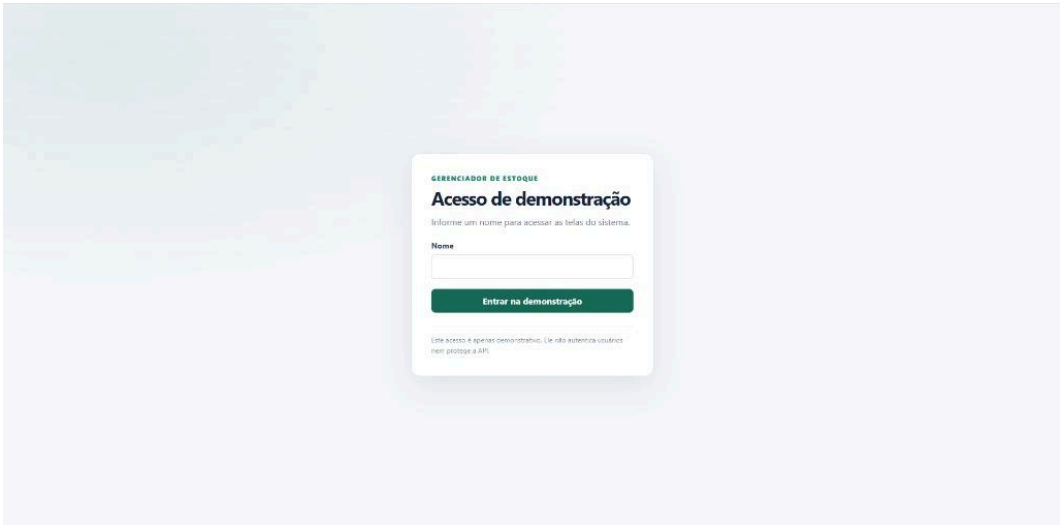


Figure 1: Tela de acesso demonstrativo para identificação do usuário

7.2. Listagem de Produtos

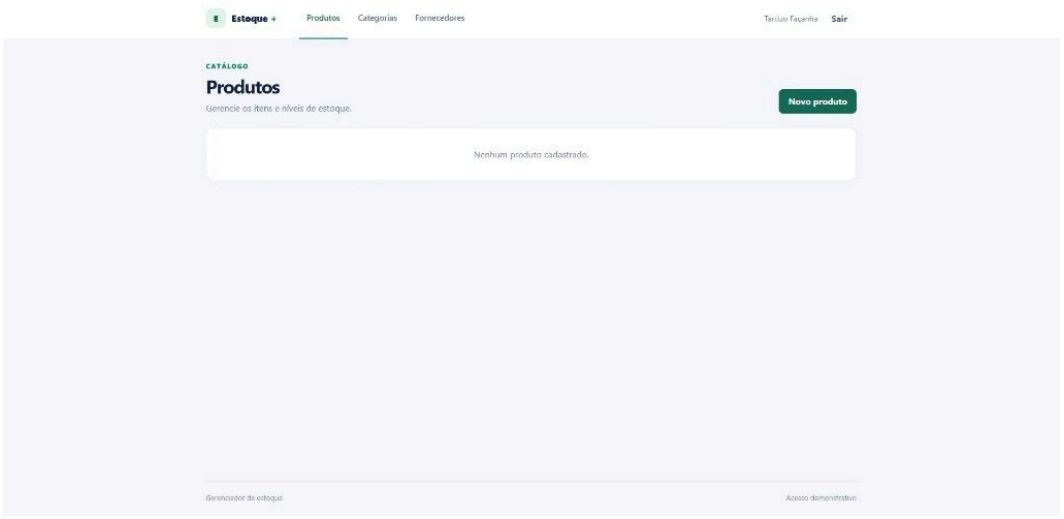


Figure 2: Tela de listagem do catálogo de produtos e níveis de estoque

7.3. Formulário de Inclusão/Edição de Produto

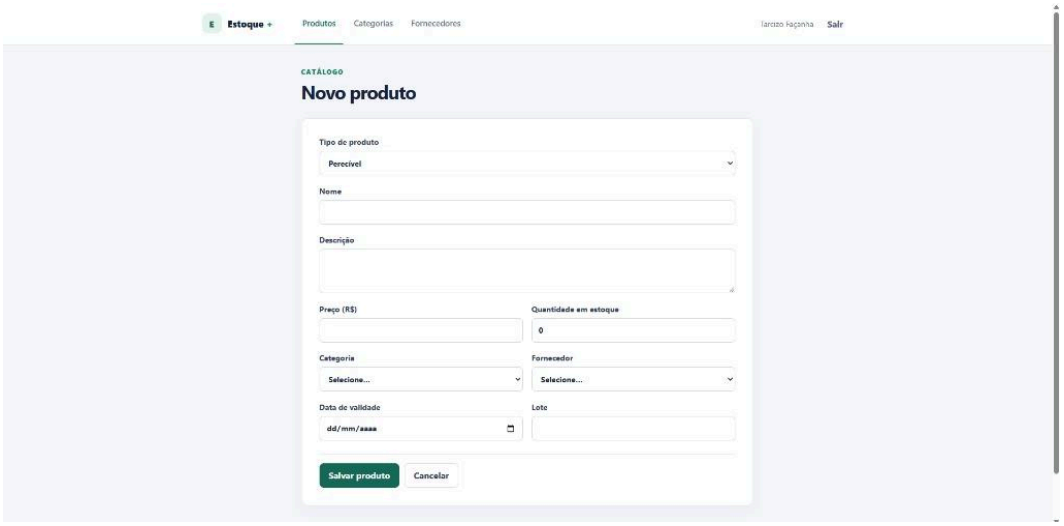


Figure 3: Formulário de produto com campos dinâmicos para produtos perecíveis e não perecíveis

7.4. Gerenciamento de Categorias

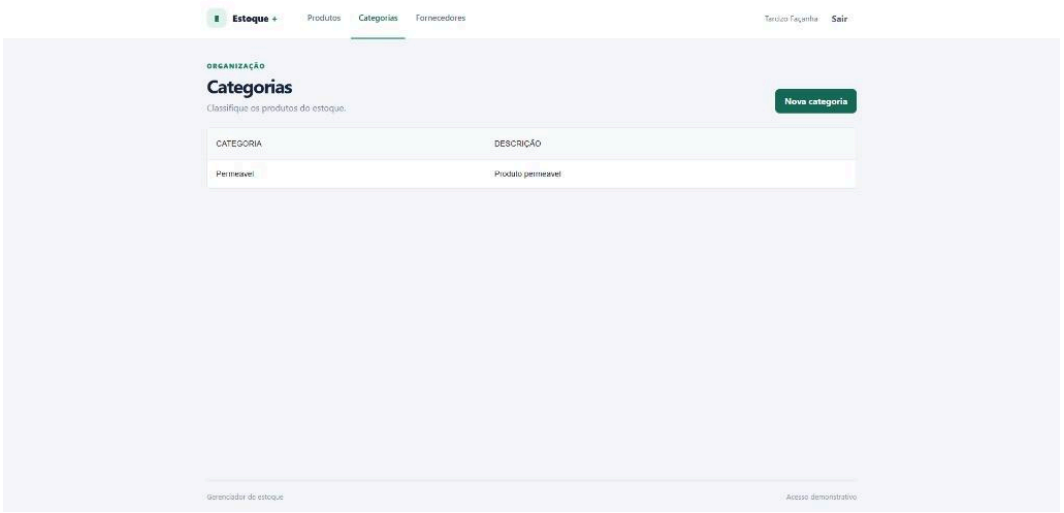


Figure 4: Tela de listagem e cadastro de categorias de produtos

7.5. Gerenciamento e Importação de Fornecedores

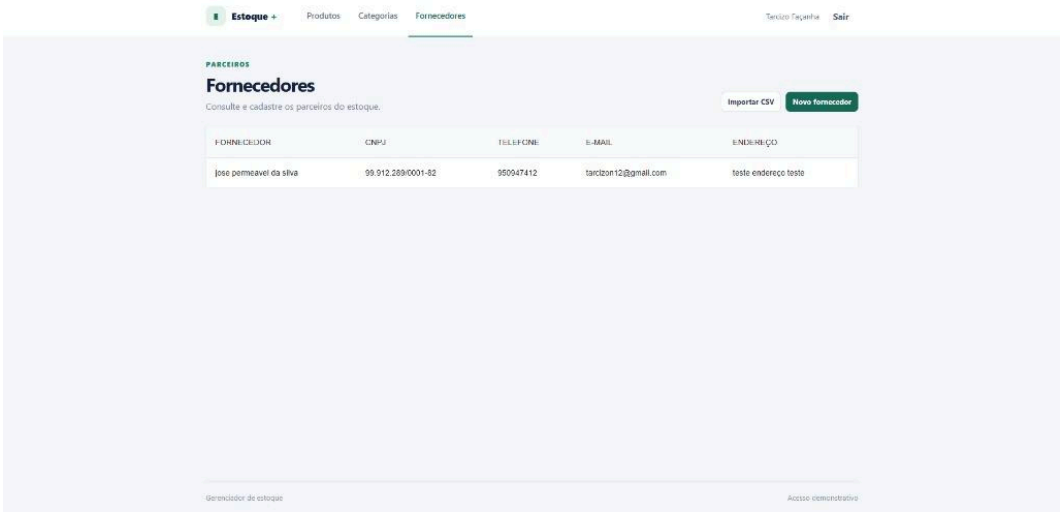


Figure 5: Listagem de parceiros fornecedores e opção de importação de arquivo CSV

7.6. Modelo de Classes e Entidades

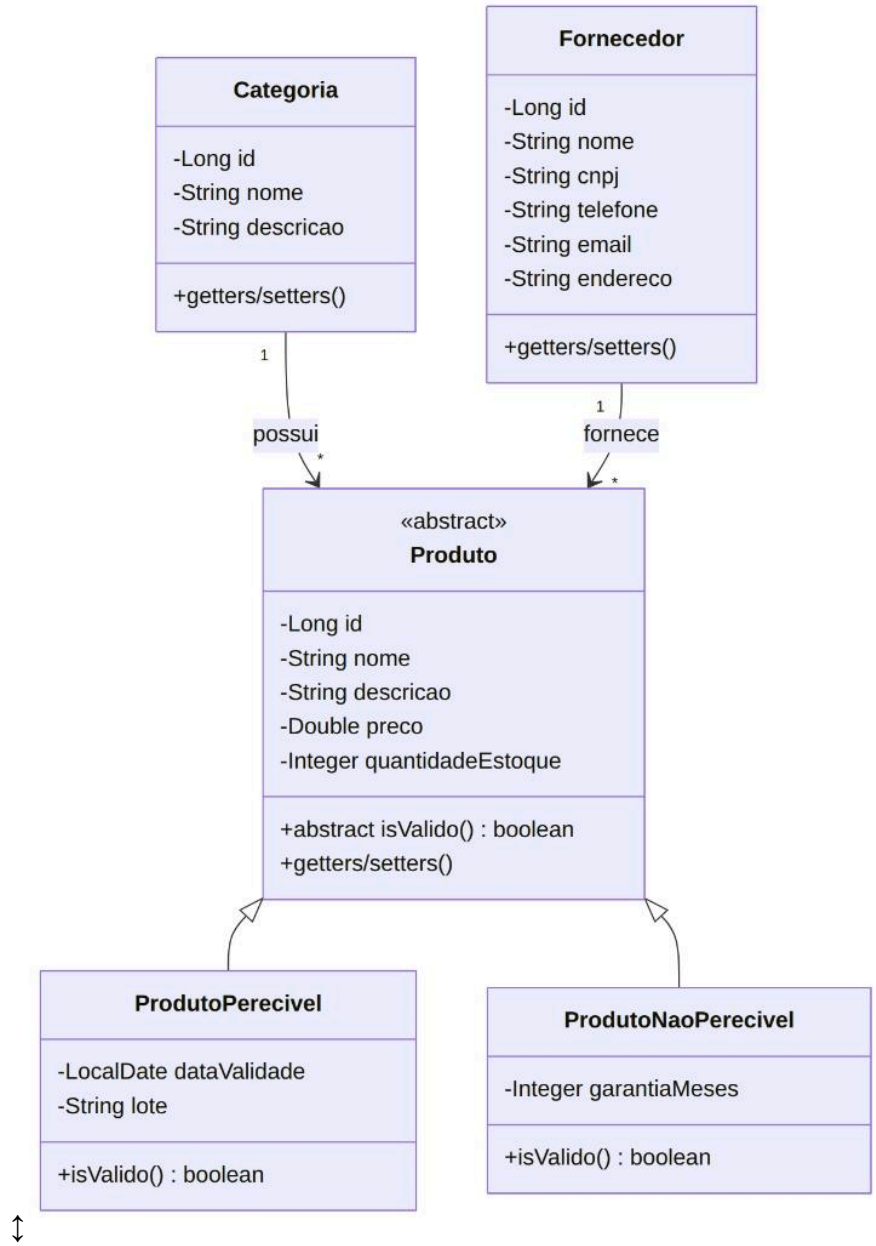


Figure 6: Diagrama de classes UML ilustrando herança da entidade Produto e associações

8. Considerações Finais

O desenvolvimento reuniu uma interface moderna em React com formulários controlados e navegação declarativa, uma API REST em Spring Boot com persistência relacional e orquestração Docker Compose para simplificar a execução completa e padronizada do ambiente.

Entre os principais desafios enfrentados e superados estiveram o alinhamento rigoroso dos contratos entre frontend e backend, a validação de regras de domínio antes da persistência,

o controle reativo de estados de carregamento e erro, e a garantia de um processo reprodutível de inicialização para avaliação.

O projeto consolidou a aplicação prática de componentes, hooks, Redux Toolkit, TanStack Query, chamadas assíncronas com tratamento de falhas, rotas demonstrativas e modelagem orientada a objetos com herança. Vale ressaltar que a rota de entrada inicial é demonstrativa e não substitui uma camada de segurança com autenticação JWT/OAuth2 em ambiente produtivo.

9. Créditos e Referências

• ViaCEP: Serviço externo consultado no formulário de cadastro de fornecedores para localização automatizada de endereço pelo CEP. A aplicação realiza chamadas GET para 'https://viacep.com.br/ws/{CEP}/json/' com 8 dígitos e trata cenários de erro e CEP inexistente.
• Correios — Desenvolvedores: Portal oficial de desenvolvedores dos Correios (https://www.correios.com.br/atendimento/developers), com referências sobre padrões de integração e endereçamento postal.
• Ecossistema Open Source: React, TanStack Query, Redux Toolkit, Material UI, Spring Framework e MySQL.

9.1. CEPs para Teste e Homologação

Na tela de cadastro de fornecedor, informe um dos CEPs válidos abaixo e selecione 'Buscar endereço':

CEP	Endereço Esperado
01001-000	Praça da Sé, Sé, São Paulo/SP
20040-020	Praça Pio X, Centro, Rio de Janeiro/RJ

O serviço espera oito dígitos; o formulário remove a pontuação antes da consulta. Para testar o tratamento de CEP inexistente, use '99999-999'; a API responde com 'erro: true', apresentado na interface como mensagem de CEP não encontrado.